

Operating Systems: Test 1

Computer Systems Overview · Memory Management · Process Management. 50 min, one letter sheet both sides.

Every formula	
Quantity	Formula
Hit ratio	$H = \text{hits} / \text{total accesses}; \text{miss} = 1 - H$
Avg access time (2-level)	$T_s = H \cdot T_1 + (1 - H) \cdot (T_1 + T_2) = T_1 + (1 - H) \cdot T_2$
Access efficiency	$T_1 / T_s = 1 / (1 + (1 - H) \cdot r), r = T_2 / T_1$
Avg cost per bit	$C_s = (C_1 \cdot S_1 + C_2 \cdot S_2) / (S_1 + S_2)$ want $C_s \leq C_2 \implies \text{need } S_1 \leq S_2$
TLB effective access time	$EAT = h(t_t + t_m) + (1 - h)(t_t + 2t_m)$
Page table size	offset bits = $\log_2(\text{page size})$ page bits = $\text{addr bits} - \text{offset bits}$ size = $2^{\text{page bits}} \times \text{entry size}$
Address space	$2^{\text{addr bits}}; \# \text{pages} = 2^{\text{addr} / 2^{\text{offset}}}$
CPU utilization	$\text{execute} / (\text{read} + \text{execute} + \text{write})$
First-fit searches	$(N + 1) / 2$ for N equally-likely blocks

Worked numbers	
Page table. 32-bit addr, byte addressing, 4 KB=2 ¹² pages, 4 B entries: 2 ³² /2 ¹² = 2 ²⁰ pages; 2 ²⁰ ×4 B = 4 MB /process. 100 processes = 400 MB.	
AMAT (average memory access time). T1=0.1 μs, T2=1 μs, H=0.95: 0.95(0.1)+0.05(1.1) = 0.15 μs , near the fast level.	
TLB. t _t =20, t _m =100 ns, h=0.8: 0.8(120)+0.2(220) = 140 ns . (Miss = page table <i>then</i> data.)	
CPU util. read 15 μs + exec 1 + write 15 = 31; 1/31 = 3.2% (why multiprogramming exists).	
Paging bits. 15-bit logical addr, 5 page bits: space 2 ¹⁵ =32 KB, 2 ⁵ =32 pages, page = 2 ¹⁰ =1 KB.	
Paging translation. 1 KB pages, logical addr 5000: page = 5000÷1024 = 4, offset = 5000–4096 = 904 . If page 4 is in frame 9, physical = 9×1024+904 = 10120 .	
Segment translation. Addr (seg 2, offset 300); seg 2 has base 4300, length 400. 300 < 400 so it is legal: physical = 4300+300 = 4600 . Offset ≥ length $\implies$ trap.	
Placement. Holes in address order 100K, 500K, 200K, 300K, 600K; request 212K , scan resuming after the 300K hole: first-fit 500K (first that fits from the start), best-fit 300K (closest in size), next-fit 600K (next that fits from where it left off).	
Two-level split. 32-bit addr, 4 KB pages: 12 offset bits, 20 page bits, split 10 10 12 . Root table = 2 ¹⁰ entries, each pointing at a 2 ¹⁰ -entry second-level table.	
Internal frag. Process 10 KB, 4 KB pages: 3 pages, the last holding 2 KB, so 2 KB wasted . Fixed partitioning: a 6 KB process in an 8 KB partition wastes 2 KB .	

I · COMPUTER SYSTEMS OVERVIEW

Basic elements

- **Processor** (CPU) controls operation + processes data. **Main memory** stores data+programs, **volatile** (= real/primary memory). **I/O modules** move data in/out. **System bus** connects all three.
- In RAM one row = **frame**; its length = **word**.
- Registers: **PC** (program counter; addr of next instr), **IR** (instruction register; the fetched instr), **MAR/MBR** (memory address/ buffer register), I/O AR + I/O BR (I/O address/buffer register).
- Variants: **microprocessor** (one chip, multi-core), **GPU** (SIMD arrays), **DSP** (streaming signals, codecs), **SoC** (all of these on one chip).

Instruction cycle

fetch $\rightarrow$ **execute** $\rightarrow$ (**interrupt check**). Fetch instr at PC, PC increments, instr lands in IR, processor interprets and acts.
Opcode: the binary code identifying an instruction.
Four action categories: processor $\rightarrow$ memory, processor $\rightarrow$ I/O, **data processing** (arith/logic), **control** (branch).

Interrupts

- **Interrupt**: lets other modules (I/O, memory, timer) break the processor's normal sequencing.
- **Why**: I/O is far slower than the CPU; without interrupts the CPU idles waiting.
- **How**: user program suspended, the **interrupt handler** (OS code) runs, process resumes *at the same point*. The user program needs **no special code**.
- An **interrupt stage** is added to the cycle; skipped if interrupts are disabled. It comes *after* execute because the current instruction must finish.

Type	Source
Program	from executing an instruction: overflow, divide by zero, illegal instruction, reference outside allowed memory
Timer	processor timer; lets the OS act at regular intervals
I/O	I/O controller: completion or error
Hardware failure	power failure, memory parity error

PSW & modes

PSW (program status word): status of the running process: condition codes, memory info, an **interrupt enable/disable bit**, a

kernel/user-mode bit. **Privileged instruction**: runs only in kernel mode.

User mode	Kernel mode
user programs run here; some memory protected, some instructions forbidden	monitor/kernel runs here; privileged instructions allowed, protected memory accessible

Evolution of OSs

Stage	What it is, and what forced the next stage
Serial processing	No OS; programmers work the hardware directly, one at a time. Scheduling and setup were awful.
Simple batch	A resident monitor (software always in memory) sequences jobs; each returns control to it. CPU still idles on I/O.
Multiprogrammed batch	Several jobs in memory; the CPU switches to another on an I/O wait. Still no user interaction.
Time sharing	Many users at terminals; the OS interleaves their programs in short bursts (quanta).

Monitor needs 4 hardware features: memory protection (user can't touch the monitor), timer (no job monopolizes), privileged instructions (monitor only), interrupts (flexibility).

	Batch multiprog.	Time sharing
Objective	maximize processor use	maximize response time
Directives	job control language with the job	commands typed at terminals

OS managers: memory, processor, device, file, network.

Uniprogramming cost: read 15 μs, execute 100 instrs = 1 μs, write 15 μs = 31 μs total, CPU busy 1/31 = **3.2%**. Multiprogramming runs another process during the waits.

Memory hierarchy

Trade-off: faster = higher cost/bit; bigger = lower cost/bit but slower. Layers, fastest first: **inboard** (registers, cache, main memory), **outboard** (disk, optical), **offline** (tape).

Going down (A–D): **A** cost/bit ↓, **B** capacity ↑, **C** access time ↑, **D** frequency of processor access ↓. **D is what makes it work**: average access time sits near the *fast* level while average cost sits near the *cheap* level.

Principle of locality: memory references **cluster**, since execution is mostly sequential and arrays/records are processed element by element. **Temporal** = same location again soon; **spatial** = nearby locations soon. Locality is why caching *and* paging work.

Secondary memory: non-volatile, external, cheaper, stores programs + data.

Cache

- Sits between CPU and main memory; **largely invisible to the OS** (all in hardware). The CPU accesses memory ≥ once per instruction cycle, so execution is limited by memory cycle time; cache hides that.
- **Design dimensions**: cache size, block size, mapping function, replacement algorithm, write policy, number of levels.
- **Block**: the unit exchanged with main memory. Bigger blocks *first raise* the hit ratio (spatial locality), then *lower* it (much of the fetch is unused, and big blocks evict useful data), so there is a sweet spot.

Mapping	Where a block may go
Direct	exactly one slot, e.g. bLock mod 10. Simplest, most conflicts.
Fully associative	any slot. Flexible, fewest conflicts, but must search many entries; costly circuitry.
Set associative	one set, any slot in it. The compromise (e.g. 10 slots as 5 sets of 2).

Replacement: LRU (least recently used) replaces the block resident longest with no references. **Write policy**: write on every update, *or* write back on replacement (fewer writes, but main memory temporarily obsolete). Write-through helps when **multiple processors share memory**: all see consistent data without coherence protocols.

II · MEMORY MANAGEMENT

Uni- vs multiprogramming

Uni: one program, whole thing loaded contiguously, memory = OS + one user program. Wasteful: size capped by memory, unused allocation wasted, CPU idle on I/O. **Multi**: divide user memory among processes so the CPU switches to another on a wait. **Memory management** decides how memory is allocated to and used by processes.

Five requirements

Relocation	Load a program into any free part of memory and still address it correctly, since its load address is unknown in advance and may change on a swap.
Protection	Stop other processes touching this process's memory; checked at run time by hardware , since the OS can't anticipate every reference.
Sharing	Let several processes reach the same memory in a controlled way (one copy of a shared library) without breaking protection.
Logical org.	Treat memory as modules so they can be compiled, protected and shared independently. Segmentation supports this best.
Physical org.	Move information between main and secondary memory: the system's job, not the programmer's.

Overlays: the old manual answer, where the programmer loads different program sections into the *same* region of memory in turn so an oversized program still runs. Virtual memory replaced it.

Types of address

Physical (absolute)	actual location in main memory
Logical	a reference <i>independent</i> of the current assignment of data to memory
Relative	a logical address expressed relative to a known point (the program origin)

Hardware translates a relative address $\rightarrow$ physical using the **base register** (start address) and **bounds register** (end address, which also enforces protection).

Why hardware? Translation runs on *every* reference, so doing it in software would add memory accesses to each one, and only hardware can bounds-check at instruction speed.

Fixed partitioning

Static partitions set at system-generation time; a process loads into a partition ≥ its size.

- **Equal-size**: placement trivial (any free partition). **Unequal-size**: assign to the **smallest partition that fits**; one queue per partition, or a single queue (better utilization, needs relocation).
- **Internal fragmentation**: space wasted *inside* a partition, since no other process can use it.
- **Problems**: internal fragmentation, inefficient memory use, a hard cap on active processes. A program bigger than the largest partition needs **overlays**.
- **Deallocation**: just reset the partition's free/busy bit.

Dynamic partitioning

Variable number and length of partitions; each process gets *exactly* what it needs. OS tracks partition boundaries.

- Kills internal fragmentation, creates **external fragmentation**: small unusable holes between processes.
- **Compaction** shifts processes together into one contiguous free block. Expensive (copies all resident processes, halts work) and **requires run-time relocation**.
- **Deallocation**: must **coalesce** adjacent free areas.

Placement	Rule / note
Best-fit	block closest in size to the request. O(n).
First-fit	first hole from the start that fits. Worst : leaves small holes near the front, forcing compaction sooner.
Next-fit	next hole that fits, scanning <i>from the last placement</i> , wrapping around. Best of the three .

Buddy system: a combination of fixed and dynamic partitioning. Blocks are always a power of two; a request takes the smallest block that fits, splitting bigger free blocks in half (the halves are **buddies**) until one is small enough, and merging a freed block with its buddy when both are free. *1 MB, request 100 KB*: 1M $\rightarrow$ 512K $\rightarrow$ 256K $\rightarrow$ 128K, allocate 128 KB (28 KB internal frag). Less external frag than dynamic, less internal than fixed, but rounds every request up to a power of 2.

Relocation: fixed vs dynamic

- **Dynamic partitioning**: relocation is **always** required (swap-in lands in a different hole; compaction shifts processes).
- **Fixed partitioning**: required **only if** a process can be reassigned to a *different* partition (single queue over unequal partitions). Always returns to its own partition $\implies$ load address never changes $\implies$ none needed.
- Either way the referenced addresses are **not fixed** $\implies$ resolve at run time via base/bounds.

Paging

- Memory in equal fixed-size **frames**; a process in equal **pages** of the same size. Pages go into any free frames, **need not be contiguous** $\implies$ **no external fragmentation**.
- Waste = internal fragmentation only, at most a fraction of the last page.
- **Transparent to the programmer**. With a power-of-2 page size the logical address = the relative address, and the physical address is formed by **appending the offset to the frame number** (no arithmetic).
- The OS keeps one **page table** per process, mapping each page number to its frame number. Free chunks = **holes**; chunks of a process = **pages**.
- A process **can't write its own page table**: it is an OS structure in protected memory, so the process must ask via a **system call** and the OS validates the change.
- **Cost**: 2 memory accesses per reference (page table, then data), so paging **doubles** access time. A page fault adds disk I/O.

Segmentation

A process split into **segments** of *varying* length (up to a max), matching logical modules. Address = (segment #, offset); the **segment table** holds each segment's **base + length**.

- **Visible** to the programmer. **Eliminates internal fragmentation**; suffers **external fragmentation**. Segments need *not* be the same length.
- Advantages: handles growing data structures; modules recompiled independently; natural sharing and protection per module.

Paging + segmentation

Split the space into segments, then each segment into pages. Translation is two-level: the segment table gives that segment's page table, which gives the frame. Combines paging's efficiency with segmentation's modularity, at the cost of two tables and two lookups.

Why paging won: it is easier to implement, and it eliminates external fragmentation with only minimal internal fragmentation. Segmentation and paging+segmentation are more complex, and segmentation still suffers external fragmentation.

Virtual memory

Virtual memory	uses secondary storage as if it were RAM; size capped by the addressing scheme + available disk , not by RAM
Virtual / real address	address in virtual memory / actual location in main memory
Virtual address space	the virtual memory assigned to one process
Resident set	the portion of a process that <i>is</i> in main memory
Page fault	the interrupt raised on referencing a non-resident page

Two breakthroughs make it possible: (1) all references are logical addresses translated dynamically at run time; (2) a process can be broken into pieces that need not be contiguous. Given both, *not all* pieces need be resident.

On a page fault: OS puts the process in **Blocked** → issues a disk read → **dispatches another process** → I/O-completion interrupt → process back to **Ready**.

Implications: more processes resident (only parts loaded) ⇒ more Ready processes ⇒ better CPU use; and a process may be **larger than all of main memory** (no programmer overlays).

Requires: hardware support for paging/segmentation, *and* OS software to move pieces between the levels.

Thrashing: the system spends most of its time swapping pieces rather than executing, because a piece is evicted just before it is needed. Avoided by **guessing from recent history** which pieces are least likely to be used soon, which works because of **locality**. The knob is **load control** (degree of multiprogramming): too many resident processes ⇒ too few frames each ⇒ thrash.

Page tables at scale

A flat table grows with the *address space* and is paid *per process*, no matter how little memory is used (4 MB each; 400 MB for 100 processes). Three fixes:

- Two-level / hierarchical:** page the page table; only the **root table is always resident**, the rest lives in virtual memory. Modern chips do 4–5 levels.
- Inverted page table:** one entry per **real frame** ⇒ size bounded by **physical memory**, one table system-wide. Indexed by frame, so a page number can't index it; a **hash** of (process id, page #) locates the entry. Entry = page #, process id, control bits, **chain pointer** (collisions).
- TLB** (translation lookaside buffer): a fast cache of recent page-table entries. A **TLB hit** gives the frame number directly, with no page-table access. All entries are compared **at once** (**associative mapping**), since a scan would cost more than it saves, so the TLB is small. Older CPUs flush it on a context switch; modern ones tag entries (ASID).

Page size

- Smaller:** less internal fragmentation, but a *bigger* page table, which may itself be paged ⇒ possible **double page fault** (page-table page, then data page).
- Larger:** better transfers from rotational secondary storage.
- Also:** main memory size and program size; OO code can *reduce* locality (many small scattered routines).
- Fault rate vs page size:** tiny pages keep much of the locality resident (few faults); growing pages hold locations further from recent references, diluting locality, so faults **rise to a max**; very large pages approach the whole process, and once one page spans it faults fall to 0. (*P* = *process size*, *W* = *working set*, *N* = *total pages*.)

OS policies

Fetch	Demand paging: bring a page in only when referenced (many faults at startup, then locality drops the rate). Prepaging: bring neighbours in too, since contiguous disk reads are cheaper per page; wasted if unreferenced. <i>Not</i> swapping.
Placement	Matters for segmentation (which hole?); irrelevant for paging, since hardware treats every frame equally.
Replacement	Evict the page least likely to be referenced soon . Frame locking: a locked frame can't be re-placed (kernel, key control structures, I/O buffers, time-critical code); one lock bit per frame.
Resident set size	Fixed (a fault replaces one of the process's own pages) vs variable (count changes over its life; more powerful, but the OS must assess behaviour).
Replacement scope	Local: only the faulting process's resident pages. Global: all unlocked frames, so another process's page can be taken. Both triggered by a fault with no free frames.
Cleaning / load control	When to write a modified page back (demand or precleaning); and the degree of multiprogramming.

Resident set trade-off: fewer frames each ⇒ more processes resident but more faults; past a point, extra frames barely change the fault rate.

Replacement algorithms

Reference string 2 3 2 1 5 2 4 5 3 2 5 2, **3 frames**:

Algorithm	Evicts	After fill	Total
Optimal	page referenced furthest in the future	3	6
LRU	page unreferenced the longest	4	7
FIFO (first in, first out)	page resident the longest , used or not	6	9
Clock	FIFO but skipping use bit = 1 (clearing it)	5	8

Optimal needs the future, so it is not implementable; it is the lower bound others are judged against. **Clock** = cheap LRU:

frames in a circular buffer with a pointer; a reference sets the use bit to 1; on a fault the pointer advances, resetting 1s to 0, and replaces the first frame already at 0. **Watch the convention:** faults that fill the empty frames count in the total but are *not* marked F in Fig. 8.14.

III . PROCESS MANAGEMENT

Program vs process

A **program** is passive (instructions in a file). A **process** is a program brought to life: an *instance running* (user view); an *entity schedulable on a processor* (scheduler view); *a unit of activity* = instructions + current state + resources (implementer view).

Elements the OS tracks (8): identifier, state, priority, program counter, memory pointers, context data, I/O status, accounting info.

PCB and process image

All of the above lives in the **PCB** (process control block). On an interrupt the PC + registers (the **context**) are saved to the PCB, the state changes, and another process runs. Because nearly every OS module reads/writes PCBs, they effectively **define the OS's state** and live in kernel memory.

PCB category	Contents
Process identification	this PID, parent PID, user id
Processor state info	user-visible registers, control/status registers, PSW , stack pointers
Process control info	state, priority, scheduling info, the event awaited , memory pointers, privileges, resources, links to other PCBs

Process image = **user program + user data + stack + PCB** (stack is LIFO, for call parameters and return addresses). Under simple schemes it is a contiguous block that must be **fully** resident; under virtual memory only part need be.

Trace and dispatcher

Trace: the sequence of instructions one process executes. The **processor's** behaviour is those traces **interleaved**. **Dispatcher:** the small program that switches the processor between processes.

A at 5000, B at 8000, C at 12000, dispatcher at 100 (6 instrs), timeout after 6, B does I/O after 4:

Instrs	Running	Stopped by
5000–5005	A	timeout
100–105	dispatcher	
8000–8003	B	I/O request, B goes to Blocked
...dispatcher, then C at 12000–12005 (timeout), dispatcher, A at 5006–5011, and so on		

Reading a trace: a **timeout** uses the whole slice and leaves the process **Ready** (dispatched again in turn); a **blocking event** stops it **early** and sends it to **Blocked** (not dispatched until the event occurs). The dispatcher's instructions are in the *processor's* trace but in **no process's** trace: switching overhead.

State models

Two-state: Running / Not Running, with a queue feeding the dispatcher. **Limitation:** "Not Running" hides **ready vs blocked**, so with one queue the dispatcher may have to *search* for a runnable process.

Five-state: New, Ready, Running, Blocked, Exit.

Transition	When
Null → New	a process is created for a program
New → Ready	OS admits it (limits on process count / committed VM)
Ready → Running	dispatch: the dispatcher selects it
Running → Ready	timeout (max time reached) or priority preemption
Running → Blocked	waits on I/O, a system call, or IPC (interprocess communication)
Blocked → Ready	the awaited event occurs
Running → Exit	completes or aborts
Ready/Blocked → Exit	a parent terminates it (or the parent itself terminates)

Queuing model: one blocked queue is simple, but on an event the OS must **scan the whole queue** for waiters (there may be thousands). Fix: a **ready queue + one blocked queue per event**.

Swapping and suspend states

Swapping moves part or all of a process out to disk (a **suspend queue**) when **no resident process is Ready**. Suspension splits in two because a **suspended process may or may not still be waiting on an event**.

Transition	When
Ready / Blocked → .../ Suspend	suspend (swap out to disk)
.../Suspend → Ready / Blocked	activate (swap back in)
Blocked/Susp. → Ready/Susp.	the event occurred while out on disk: runnable, but not resident
Running / New → Ready/Susp.	suspend the running process; or admit a new one straight to disk

Given a choice the OS **activates Ready/Suspend**, since a Blocked/Suspend process would occupy memory and still have to wait.

A **suspended process (4)**: not immediately available to run; may or may not be waiting on an event; was placed there by an **agent** (itself, a parent, or the OS); stays until that agent releases it.

Reasons to suspend (5): swapping (free memory for a ready process); other OS reason (background/suspect process); inter-

active user request (debugging); timing (periodic process between intervals); parent process request.

OS control tables

Four cross-referencing groups: **memory tables** (real + virtual allocation, protection attributes, VM info), **I/O tables** (device status, buffers), **file tables** (existence, location, status), **process table** (the PCBs). They must cross-reference: the OS reaches a process's resources from its PCB and the owner from a resource.

Modes and the kernel

User mode (less privileged) vs **system/kernel mode** (more privileged; privileged instructions; runs the **kernel** = the core of the OS). This protects the OS and its tables (PCBs, page tables). The **PSW's mode bit** flips to kernel on a system call or interrupt and back on the interrupt return. x86 has 4 rings: **ring 0 = kernel, ring 3 = user**.

Kernel functions: *process management* (creation/termination, scheduling, dispatching, switching, synchronization + IPC, PCB management); *memory management*; *I/O management*; *support* (interrupt handling, accounting, monitoring).

Process creation and termination

Five steps: (1) assign a unique **PID** (process identifier); (2) allocate space; (3) initialise the **PCB**; (4) set the right **linkages** (e.g. into a queue); (5) create or expand other data structures (e.g. accounting).

Causes: new batch job; interactive log-on; created by the OS for a service (e.g. printing, so the user needn't wait); **spawned** by an existing process, which makes it the **parent** and the new one the **child**.

Termination: halt instruction or an OS service call (raises an interrupt); log-off / terminal shutdown; quitting an application; an error or fault condition.

Interrupting a process

Mechanism	Cause	Use
Interrupt	external to the current instruction	asynchronous external event: clock, I/O, memory fault
Trap	associated with the current instruction	error/exception inside the running process. The OS decides if it is fatal; if so, the process moves to Exit and a process switch follows.
Supervisor call	explicit request	a call to an OS function

Time slice: the max time a process may execute before being interrupted.

Mode switch vs process switch

Mode switch is cheaper: if an interrupt is pending the processor sets the PC to the handler's address and switches from user to kernel mode; if the same process is still best, its context is restored and the **running process never changed**. (No interrupt pending: just fetch the next instruction.)

Process switch, 7 steps: (1) save the processor context; (2) update the PCB of the process leaving Running; (3) move that PCB to the right queue (Ready / Blocked / Ready-Suspend); (4) select another process; (5) update *its* PCB; (6) update memory-management structures; (7) restore that process's saved context.

Traps & distinctions

- Internal vs external frag.** Internal = waste *inside* an allocation (fixed partitioning, last page). External = unusable holes *between* allocations (dynamic partitioning, segmentation).
- Logical vs relative vs physical.** Relative is a *kind of* logical address (from the program origin). The processor translates relative into physical, not the reverse.
- Paging vs segmentation.** Paging: equal fixed sizes, invisible to the programmer, internal frag. Segmentation: variable sizes, visible, external frag.
- Page vs frame.** Pages are chunks of a *process*; frames are chunks of *memory*. Pages need not land in contiguous frames.
- Swapping vs prepaging.** Swapping moves a process out to disk; prepaging brings extra pages *in*.
- Interrupt vs trap.** External to the instruction vs caused by it.
- Mode switch vs process switch.** A mode switch need not change the running process.
- Blocked vs Ready.** Blocked waits for an *event*; Ready waits only for the *processor*. A timeout leaves a process Ready.
- Main vs secondary memory.** Main is volatile, fast, expensive. Secondary is non-volatile, slow, cheap, *not* "fast access at relatively high cost".
- Relocation under fixed partitioning** is required only *sometimes*; under dynamic partitioning, *always*.